Cognitive Modeling

Complete Lecture Notes

Week 13: Probabilistic Programming and Hierarchical Models

Contents

1 Parameter Estimation with MCMC 1

1.1 Week 13: Probabilistic Programming and Hierarchical Models 1

1.1.1 Key terms 1

1.1.2 Learning outcomes 2

1.1.3 Motivation & big picture 3

1.1.4 Lecture notes 4

1.1.5 Worked example 8

1.1.6 Lab (2 hours) 8

1.1.7 Connection to AI & LLMs 11

1.1.8 Common mistakes & misconceptions 12

1.1.9 Check your understanding 13

1.1.10 Assignment 13

1.1.11 Further reading 14

Chapter 1

Parameter Estimation with MCMC

1.1 Week 13: Probabilistic Programming and Hierarchical Models

> Chapter 5: Parameter estimation with data-driven methods (MCMC), bridging to Chapter 6 · Lecture 1 h · Lab 2 h Running data: a recognition/detection experiment with 12 participants and only 50 + 50 trials each: hierarchical SDT in PyMC.

1.1.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
hierarchical model	مدل سلسله‌مراتبی (چندسطحی)	پارامترهای فردی از یک توزیع گروهی می‌آیند
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
estimate / estimator	برآوردگر / برآورد	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
trial	کوشش (trial)	یک بار انجام تکلیف توسط شرکت‌کننده
probabilistic programming	برنامه‌نویسی احتمالاتی	تعریف مدل با کد و نمونه‌گیری خودکار (PyMC)
prior	توزیع پیشین (prior)	باور درباره‌ی پارامتر پیش از دیدن داده
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها

English	فارسی	توضیح کوتاه
prior predictive check	بررسی پیش‌بینی پیشین	شبیه‌سازی داده از پیشین برای سنجش معقول بودن آن
posterior	توزیع پسین (posterior)	باور درباره‌ی پارامتر بعد از دیدن داده
posterior predictive check (PPC)	بررسی پیش‌بینی پسین	مقایسه‌ی داده‌ی شبیه‌سازی‌شده از پسین با داده‌ی واقعی
HMC / NUTS	Hamiltonian Monte Carlo / NUTS	MCMC مبتنی بر گرادیان
divergent transition	divergence (گذار واگرا)	هشدار NUTS برای ناحیه‌ای که درست کاوش نشده
complete / no / partial pooling	ادغام کامل / بدون ادغام / ادغام جزئی (pooling)	جدا برای هر فرد / یک پارامتر برای همه / سلسله‌مراتبی
shrinkage	انقباض به سمت میانگین (shrinkage)	کشیده شدن برآوردهای پرنویز به سمت میانگین گروه
centered / non-centered parameterization	پارامتری‌سازی centered / non-centered	دو شکل نوشتن مدل سلسله‌مراتبی
group-level parameter	پارامتر سطح گروه	میانگین و پراکندگی پارامترها در جمعیت
fit / fitting	برازش	تنظیم پارامترها تا مدل با داده جور شود
hit	اصابت (hit)	پاسخ «بله» وقتی علامت وجود دارد
sensitivity (d′)	حساسیت	فاصله‌ی توزیع علامت و نویز
gradient	گرادیان	جهت و شیب تغییر تابع
simulation	شبیه‌سازی	تولید داده‌ی مصنوعی با اجرای مدل
reparameterization	بازپارامتری‌سازی	نوشتن پارامتر در مقیاس دیگر، مثلاً $\lambda = e\hat{}\eta$
standard error (SE)	خطای استاندارد	پراکندگی برآورد در تکرارهای فرضی آزمایش
benchmark	benchmark (مجموعه‌آزمون استاندارد)	مجموعه سؤال برای مقایسه‌ی مدل‌های هوش مصنوعی

1.1.2 Learning outcomes

By the end of this week, students will be able to:

1. Specify a cognitive model in PyMC (priors, deterministic transformations, likelihood) and run a complete Bayesian workflow: prior predictive check, sampling, diagnostics, posterior predictive check.
2. Explain conceptually how Hamiltonian Monte Carlo / NUTS differs from random-walk Metropolis, and what a divergence signals.

3. Formulate a hierarchical (multilevel) model and explain partial pooling and shrinkage, contrasting it with the complete- and no-pooling approaches of Week 5.
4. Implement a non-centered parameterization and justify its use.
5. Interpret group-level and individual-level posteriors and report them according to the Week 12 checklist.

1.1.3 Motivation & big picture

In Week 5 we faced a dilemma. Fitting the average of all participants is stable but can distort the model (aggregation artifacts) and ignores individual differences. Fitting each participant separately respects differences, but with few trials per person the estimates are noisy. Some become extreme or infinite (a participant with 50/50 hits has $d' = \infty$ without a correction). The second-stage statistics on those estimates also ignore that some are much less certain than others.

Hierarchical models dissolve the dilemma. They assume that each participant has their own parameter, *and* that these individual parameters come from a group-level distribution whose mean and spread are also estimated. Each individual estimate then borrows strength from the others: noisy, extreme estimates are pulled ("shrunk") toward the group mean, by exactly as much as their uncertainty warrants. The group-level parameters answer the population question ("what is the typical sensitivity, and how much do people differ?") with all sources of uncertainty included.

Writing samplers for such models by hand would be a research project. Probabilistic programming languages like PyMC let us *describe* the model in a few lines and use a state-of-the-art sampler, NUTS, automatically. The hand-built samplers of Weeks 11–12 are what make this black box transparent to you.

Where is this used?

- Most modern cognitive modeling studies with many participants fit hierarchical models: hierarchical drift-diffusion models (the HDDM package), hierarchical reinforcement-learning models in computational psychiatry, hierarchical SDT in memory research.
- Clinical and developmental studies with short sessions: children and patients cannot do hundreds of trials, so individual estimates are noisy; partial pooling makes them usable.
- Education and social science: students within classes within schools (multilevel models) are the textbook example of hierarchical structure.
- Sports analytics and medicine: estimating a player's skill or a hospital's success rate from few observations by borrowing strength from the group is the same idea.

1.1.4 Lecture notes

In plain words When each person gives us only a little data, their individual estimates are noisy, and some come out absurd (a perfect score gives $d' = \infty$). A hierarchical model says: each person has their own parameter, *but* people are similar to each other, so their parameters come from a common group distribution, whose mean and spread we also estimate. The effect is sensible compromise: a person with little or extreme data is pulled toward the group average (shrinkage), and a person with plenty of data keeps an estimate close to their own. How strong the pull is depends on how different people turn out to be, which is learned from the data. Writing a sampler for such a model by hand is hard, so we use PyMC: we write the model almost like the math, and PyMC runs an advanced sampler (NUTS) for us. The last part of the week is a step-by-step workflow for doing this safely.

Suggested timing (60 min): probabilistic programming 10 · HMC/NUTS concepts 10 · hierarchical models 20 · parameterization and divergences 10 · the Bayesian workflow 10.

1. Probabilistic programming with PyMC

A PyMC model is a direct transcription of the mathematical model:

Math	PyMC
$\mu_d \sim \mathcal{N}(1, 1)$	`mu_d=pm.Normal("mu_d",1,1)`
$\sigma_d \sim \mathcal{N}^+(0, 1)$	`sigma_d=pm.HalfNormal("sigma_d", 1)`
$d'_j = \mu_d + \sigma_d z_j$	`d=pm.Deterministic("d",mu_d+sigma_d*z_d)`
$H_j = \Phi(d'_j/2 - c_j)$	`H=Phi(d/2-c)` (an expression, built with `pm.math`)
$\text{hits}_j \sim \text{Binomial}(n_s, H_j)$	`pm.Binomial("hits",n=n_s,p=H,observed=hits)`

Then `pm.sample()` compiles the model, computes gradients automatically, and runs NUTS. The result is an ArviZ `InferenceData`/`DataTree` object with groups such as `posterior`, `sample_stats`, `prior_predictive` and `posterior_predictive`, which ArviZ summarizes and diagnoses (Week 12).

Practical notes: use `cores=1` when running a plain `.py` script on Windows (or protect the code with `if__name__=="__main__":`); give every variable a unique name; use `dims`/`coords` to label participants.

2. Hamiltonian Monte Carlo and NUTS (conceptually)

Random-walk Metropolis proposes blind steps and wastes most of them in high dimensions or along correlated ridges (Weeks 11–12). Hamiltonian Monte Carlo (HMC) treats the negative log posterior as a landscape and the current parameter vector as a frictionless ball. Give the ball a random push (momentum), let it roll for a while following the landscape's gradient, and propose where it ends

up. Proposals can be far away yet still in high-probability regions, so acceptance is high and autocorrelation low. NUTS (the No-U-Turn Sampler) tunes the rolling time automatically. HMC needs gradients, which PyMC computes for you.

Divergences. The rolling is simulated numerically in small steps. Where the posterior's curvature changes abruptly (typically in "funnels", below), the simulation becomes inaccurate and NUTS flags a divergent transition. Even a few divergences mean the posterior is not being explored reliably there. Do not ignore them; reparameterize.

> Advanced: HMC simulates Hamiltonian dynamics with energy $H(\boldsymbol{\theta}, \mathrm{p}) = -\ln \tilde{p}(\boldsymbol{\theta}) + \frac{1}{2}\mathrm{p}^\top M^{-1}\mathrm{p}$ using the leapfrog integrator. Exact dynamics conserve H, so proposals would always be accepted. The leapfrog error makes a Metropolis correction necessary, and a divergence is a trajectory whose energy error explodes.

3. Hierarchical models

Structure (for SDT with J participants):

$$
\begin{aligned}
&\text{Group level:} && \mu_d \sim \mathcal{N}(1,1),\ \ \sigma_d \sim \mathcal{N}^+(0,1),\ \ \mu_c \sim \mathcal{N}(0,1),\ \ \sigma_c \sim \mathcal{N}^+(0,1)\\
&\text{Individual level:} && d'_j \sim \mathcal{N}(\mu_d, \sigma_d),\ \ c_j \sim \mathcal{N}(\mu_c, \sigma_c),\quad j = 1, \ldots, J\\
&\text{Data:} && \text{hits}_j \sim \text{Binomial}\big(n_s, \Phi(d'_j/2 - c_j)\big),\ \ \text{FAs}_j \sim \text{Binomial}\big(n_n, \Phi(-d'_j/2 - c_j)\big)
\end{aligned}
$$

Three ways to pool information (cf. Week 5):

Approach	Model	What it assumes
Complete pooling	One d' and one c for everyone	$\sigma_d = \sigma_c = 0$: no individual differences
No pooling	Separate d'_j, c_j with fixed, independent priors	$\sigma \to \infty$: participants tell us nothing about each other
Partial pooling	Hierarchical; σ_d, σ_c *estimated*	The data decide how similar participants are

Shrinkage. For a normal approximation, the hierarchical estimate of participant j is a precision-weighted average of their own estimate $\hat{d}_j$ (with standard error SE_j) and the group mean:

$$\mathbb{E}[d'_j \mid \text{data}] \approx \frac{\hat{d}_j/\text{SE}_j^2 + \mu_d/\sigma_d^2}{1/\text{SE}_j^2 + 1/\sigma_d^2}.$$

Participants with noisier data (larger SE_j) are shrunk more. If people really differ a lot (large σ_d), shrinkage is weak; if they are similar, it is strong. This is learned from the data.

Why shrinkage is good, not a distortion: across participants, shrunken estimates are on average *closer to the true values* than the raw individual estimates. This is a famous result in statistics (Stein's para-

dox). Extreme raw estimates are partly extreme by chance, and shrinkage corrects for that regression to the mean.

Group-level questions are answered directly by the posteriors of μ_d (typical sensitivity) and σ_d (individual differences), and group comparisons are made by adding group or condition effects to the group level.

Reading the math

- Two levels. Group level: μ_d (typical sensitivity) and σ_d (how much people differ). Individual level: $d'_j \sim \mathcal{N}(\mu_d, \sigma_d)$, person j's sensitivity is drawn from the group distribution. Data level: the hits and false alarms of person j depend only on that person's d'_j and c_j.
- $\mathcal{N}^+(0,1)$: a "half-normal" distribution, a normal curve folded at 0 so only positive values remain (a standard deviation cannot be negative).
- The shrinkage formula is a precision-weighted average (as in Week 12): the person's own estimate $\hat{d}_j$ has precision $1/\text{SE}_j^2$, the group distribution has precision $1/\sigma_d^2$. Example from the worked example: SE $= 0.40$ gives precision 6.25, the group gives 4, so the estimate is $(6.25 \times 2.4 + 4 \times 1.3)/10.25 = 1.97$: the raw 2.4 moves 40% of the way toward the group mean 1.3.
- If people really differ a lot (large σ_d), the group precision is small and shrinkage is weak. If they are similar, shrinkage is strong.

4. Centered vs. non-centered parameterization

Writing $d'_j \sim \mathcal{N}(\mu_d, \sigma_d)$ directly (the centered form) creates a "funnel" in the posterior: when σ_d is small, all d'_j must be squeezed close together, and when it is large they can spread out. NUTS struggles with this changing geometry and produces divergences, especially with few data per participant. The non-centered form samples standardized deviations and builds d'_j deterministically:

$$z_j \sim \mathcal{N}(0,1), \qquad d'_j = \mu_d + \sigma_d z_j.$$

It describes the same model but has a much friendlier geometry. Use it by default when data per participant are sparse.

Reading the math $z_j \sim \mathcal{N}(0,1)$, $d'_j = \mu_d + \sigma_d z_j$: instead of sampling d'_j directly, sample a standardized "how unusual is this person" score z_j (in units of the group SD) and rebuild d'_j from it. Both versions describe exactly the same model. The difference is only in what the sampler has to explore: in the centered form, the allowed range of every d'_j shrinks to almost nothing when σ_d is small (a funnel); in the non-centered form, the z_j always live on the same scale, so the sampler's step size fits everywhere.

5. The Bayesian workflow

1. Specify the model and priors (with justification).
2. Prior predictive check (Week 9): does the prior imply plausible data?

3. Fit with several chains.
4. Diagnose: $\hat{R}$, ESS, divergences, trace plots (Week 12).
5. Posterior predictive check (Week 10): can the fitted model reproduce the data?
6. Compare models if needed (LOO via ArviZ, Bayes factors; Weeks 6, 10).
7. Interpret and report.

For an engineer

- Hierarchical model = estimation with a population prior. It is like estimating each user's channel gain from a few pilots while also learning the distribution of gains in the cell, and using it as the prior: an empirical-Bayes / MMSE estimator. Shrinkage toward the group mean is the same phenomenon as the James–Stein estimator, which beats separate ML estimates in total squared error when there are three or more units.
- HMC/NUTS simulates Hamiltonian dynamics (a frictionless particle on the surface −log posterior) with a numerical integrator (leapfrog), as in physics simulations. A divergence is the integrator becoming unstable because the step size is too large for the local curvature, like a stiff ODE integrated with a too-large step.
- Non-centered parameterization is a change of variables that normalizes the variables to unit scale, i.e. it improves the conditioning of the problem, much like whitening or normalizing inputs before an adaptive algorithm.
- PyMC computes gradients automatically (automatic differentiation), the same technology that trains neural networks.

For the instructor

- Introduce shrinkage with a sports or exam example. A student who answered 3 of 3 questions correctly is probably not a "100% student"; a student with 95 of 100 probably is near 95%. Ask the class how they would combine individual and class information: they invent partial pooling themselves.
- Show the shrinkage plot from lab Part D: raw estimates, hierarchical estimates and the group mean on one figure, with arrows. It makes the idea stick.
- Do not teach HMC mathematics in depth. One picture (a ball rolling on the negative log posterior, given a random push) is enough; the important practical skill is reading the diagnostics (divergences, $\hat{R}$, ESS).
- Running PyMC in class: on Windows use `cores=1` in scripts, and start sampling early (Part C), since it can take a few minutes.
- Typical questions.
  - *"Isn't shrinkage biasing the results?"* Slightly, but it reduces the total error, so estimates are closer to the truth on average.
  - *"Is a hierarchical model always better?"* When individuals have enough data, results are similar to separate fits; the gain is largest with little data per person.
  - *"What does a divergence mean for my results?"* That part of the posterior was not explored reliably. Reparameterize and rerun until there are none.

1.1.5 Worked example

Shrinkage by hand. The group posterior has $\mu_d \approx 1.3$ and $\sigma_d \approx 0.5$ (group precision $1/0.5^2 = 4$). Two participants both have a raw estimate $\hat{d}_j = 2.4$.

Participant	SE_j	Individual precision $1/SE_j^2$	Hierarchical estimate	Posterior SD
P1 (few trials)	0.40	6.25	$(6.25 \times 2.4 + 4 \times 1.3)/(6.25 + 4) = 20.2/10.25 = 1.97$	$1/\sqrt{10.25} = 0.31$
P2 (many trials)	0.20	25	$(25 \times 2.4 + 4 \times 1.3)/(25 + 4) = 65.2/29 = 2.25$	$1/\sqrt{29} = 0.19$

Interpretation. Both participants *look* equally sensitive, but P1's estimate rests on less data, so the model trusts it less and moves it 0.43 toward the group mean. P2 moves only 0.15. Both posterior SDs are smaller than the raw SEs, because the group distribution adds information. (In the full model, μ_d and σ_d are themselves uncertain and estimated jointly; this calculation fixes them for illustration.)

1.1.6 Lab (2 hours)

Goals

1. Simulate a 12-participant experiment and compute individual (no-pooling) estimates.
2. Write the hierarchical SDT model in PyMC with a non-centered parameterization; run a prior predictive check.
3. Sample with NUTS; check $\hat{R}$, ESS and divergences.
4. Visualize shrinkage and compare the estimates with the true values.
5. Run a posterior predictive check and interpret the group-level parameters.

Timing: Part A 10 min · Part B 20 min · Part C 25 min · Part D 30 min · Part E 20 min · Part F 15 min.

> Run time: sampling takes from under a minute (conda-forge install with a compiler) up to about 2–3 minutes (no compiler). Start Part C early.

Part A: data and no-pooling estimates (10 min)

> Code · کد: `code/week13.py`, cell 1 · سلول 1

Expected output:

```
    hits  fas  d_true  d_indiv  c_true  c_indiv
0     46    2    2.21     3.01    0.13     0.15
1     16   25   -0.24    -0.46    0.42     0.23
2     28    6    1.78     1.29    0.41     0.50
3     40   11    1.33     1.57    0.05    -0.03
4     43    9    1.96     1.94    0.09    -0.08
5     43   17    1.49     1.45   -0.34    -0.32
6     31    0    2.21     2.63    0.70     1.02
7     35   11    1.32     1.27    0.13     0.12
8     24    8    1.04     0.92    0.60     0.51
9     39    8    1.59     1.72    0.33     0.11
10    27    3    1.52     1.58    0.78     0.69
11    21    9    1.12     0.69    0.66     0.54
```

Part B: the model in PyMC, and a prior predictive check (20 min)

> Code · کد: `code/week13.py`, cell 2 · سلول 2

Expected output:

```
Share of prior-predictive rates exactly 0 or 1: 0.30
```

The prior implies a broad range of hit and false-alarm rates, mostly above the diagonal. About 30% of simulated participants have a hit or false-alarm count of exactly 0 or 50. These are mostly participants simulated with very high sensitivity and a strong bias, which produce zero false alarms. This is far better than the 96% of the "vague" prior in Week 9, but still on the generous side: tightening the priors on `sigma_d` and `sigma_c` (e.g., `HalfNormal(0.5)`) would remove most of them. Try it and check whether the posterior changes.

Part C: sampling and diagnostics (25 min)

> Code · کد: `code/week13.py`, cell 3 · سلول 3

Expected output:

```
        ess_bulk ess_tail r_hat mcse_mean mcse_sd
mu_d        1303     1651  1.00    0.0075  0.0054
sigma_d     1639     2088  1.00    0.0055  0.0045
mu_c        1312     1474  1.00    0.0036   0.003
sigma_c     1203     1748  1.00    0.0034  0.0047
Number of divergent transitions: 0
Max R-hat over all individual d and c: 1.0023234826538692
```

Part D: shrinkage and group-level inference (30 min)

> Code · کد: `code/week13.py`, cell 4 · سلول 4

Expected output:

```
RMSE vs. true d':  no pooling = 0.339,  hierarchical = 0.279
mu_d     posterior mean = 1.45, 95% interval = [0.90, 1.95]
sigma_d  posterior mean = 0.90, 95% interval = [0.56, 1.42]
mu_c     posterior mean = 0.29, 95% interval = [0.04, 0.55]
sigma_c  posterior mean = 0.38, 95% interval = [0.22, 0.65]
P(mu_c > 0 | data) = 0.987
```

Interpret (write in your notebook): 1. Which participants are shrunk most, and why? Find one whose raw estimate is extreme and describe what the hierarchical model does with it. 2. Is the hierarchical estimate closer to the truth on average? Is it closer for *every* participant? Why is that not required? 3. What do μ_d and σ_d tell you about this population? Compare σ_d with the SD used to simulate the data (0.5) and explain the difference (hint: 12 participants).

Part E: posterior predictive check (20 min)

> Code · کد: `code/week13.py`, cell 5 · سلول 5

Expected output:

```
Share of observed counts inside their 95% predictive interval: 1.00
```

Part F: report (15 min)

Write a methods-and-results paragraph following the Week 12 checklist: model and priors (in words and symbols), sampler and settings, diagnostics (max $\hat{R}$, min ESS, divergences), the PPC result, and the group-level conclusions (μ_d, σ_d, μ_c with 95% intervals; $P(\mu_c > 0 \mid \text{data})$).

Exercises (increasing difficulty)

Exercise 1 ⋆. Fit the complete-pooling model (one d' and one c for everyone) in PyMC. Compare its estimate of d' with μ_d from the hierarchical model, and its posterior predictive check with Part E. Which participants does it fail to predict?

Exercise 2 ⋆. Rerun Part C with the centered parameterization (`d=pm.Normal("d",mu_d,sigma_d,dims="participant")`). Compare divergences, ESS and run time. Then reduce the data to 10 + 10 trials per participant and compare again.

Exercise 3 ⋆⋆ (unequal trial numbers). Give half of the participants 20 + 20 trials and the other half 200 + 200 (simulate new data). Show that shrinkage depends on the number of trials, as in the worked example.

Exercise 4 ⋆⋆ (model comparison with LOO). Compute the log-likelihood with `pm.compute_log_likelihood(idata)`. ArviZ's `loo` needs one pointwise log-likelihood per observation: add the `hits` and `fas` log-likelihood arrays into a single per-participant array (e.g., `idata.log_likelihood["obs"]=idata.log_likelihood["hits"]+idata.log_likelihood["fas"]`) and call `az.loo(idata,var_name="obs")`. Compare the hierarchical and complete-pooling models with `az.compare`. Check the Pareto-k diagnostics and say what they mean for leave-one-participant-out prediction.

Exercise 5 ⋆⋆⋆ (Advanced: a group effect). Simulate two groups of 12 participants that differ in mean sensitivity by 0.4. Extend the model with a group effect β on μ_d ($d'_j = \mu_d + \beta \cdot \text{group}_j + \sigma_d z_j$), estimate β, and report $P(\beta > 0 \mid \text{data})$. Compare with a t-test on the individual d' estimates.

Lab check-off

Show: (1) the diagnostics output (no divergences, $\hat{R}$, ESS), (2) the shrinkage plot with the RMSE comparison, and (3) the PPC plot with your Part F paragraph.

Project progress check (this week): submit a one-page status report (what is done, a first fit or simulation figure, open problems). It is not graded; written feedback is returned within a week.

1.1.7 Connection to AI & LLMs

Partial pooling for AI benchmarks. Benchmarks are split into task categories, and some categories have only a handful of items. Reporting raw accuracy per category gives noisy, sometimes extreme numbers, just like individual SDT fits with few trials. A hierarchical model treats category accuracies as draws from a common distribution and shrinks small categories toward the overall mean. The code below uses a fast empirical-Bayes version, a beta-binomial model whose group parameters are estimated by maximizing the marginal likelihood on a grid. In the exercises you fit the full Bayesian version in PyMC.

Code · کد: `code/week13.py`, cell 6 · سلول 6

Expected output:

```
items   true    raw  shrunk
    5  0.72  0.80   0.71
    8  0.63  0.88   0.74
   10  0.64  0.50   0.63
   15  0.63  0.67   0.68
   25  0.76  0.88   0.80
   40  0.69  0.55   0.60
   60  0.80  0.78   0.76
   80  0.51  0.57   0.60
group mean = 0.69, concentration = 19.9;  RMSE raw = 0.125, shrunk =
 ↪  0.066
```

The 8-item category (raw 0.88, true 0.63) is pulled down strongly, and the shrunken estimates are about twice as accurate overall (RMSE 0.066 vs. 0.125).

Try it (no API needed). - ⋆ Which categories move most, and why? Relate your answer to the shrinkage formula in this week's lecture notes. - ⋆⋆ Fit the same model in PyMC: `mu~Beta(2,2)`, `kappa~Gamma(2,0.1)` (shape and rate), `theta_j~Beta(mu*kappa,(1-mu)*kappa)`, `y_j~Binomial(n_j,theta_j)`. Compare the posterior means with the empirical-Bayes estimates, and check the diagnostics. - ⋆⋆ Discussion: a leaderboard highlights "the category where Model X is weakest", a category with 5 items. What would you tell the authors?

1.1.8 Common mistakes & misconceptions

1. "Shrinkage biases the results." It trades a little bias for a large reduction in variance and gives more accurate estimates on average. It is principled, not an ad-hoc correction.
2. Ignoring divergences. Even a handful indicate unreliable exploration. Reparameterize (non-centered), tighten priors on scale parameters, or increase `target_accept`.
3. Centered parameterization with sparse data. It invites funnels and divergences. Default to non-centered for hierarchical models with few trials per participant.
4. Vague priors on group SDs. Very wide priors on σ can produce implausible prior predictions and sampling problems. Half-normal priors on a sensible scale are usually better.
5. Treating the black box as magic. PyMC samples whatever model you wrote, including wrong ones. The workflow (prior predictive check, diagnostics, PPC) still applies in full.
6. Running multi-process sampling from a script on Windows without a main guard, which causes cryptic multiprocessing errors. Use `cores=1` or `if__name__=="__main__":`.
7. Reporting only individual estimates. The group-level parameters often answer the actual research question.

1.1.9 Check your understanding

1. What are the three ways to combine data across participants, and what does each assume about individual differences? Answer Complete pooling (everyone identical, $\sigma = 0$), no pooling (participants unrelated, so no information is shared), and partial pooling/hierarchical (participants come from a common distribution whose spread is estimated from the data).

2. Two participants have the same raw d' estimate. Why may the hierarchical model shrink one of them much more? Answer Shrinkage depends on the precision of each individual estimate. The participant with fewer trials (larger SE) carries less information and is pulled more strongly toward the group mean.

3. Why is the non-centered parameterization often preferred? Answer The centered form creates a funnel-shaped posterior (individual parameters must collapse together when the group SD is small) that NUTS explores poorly, causing divergences. Sampling standardized z_j and computing $d'_j = \mu + \sigma z_j$ gives a geometry that is much easier to explore.

4. What is a divergent transition, and what should you do about it? Answer A NUTS trajectory whose numerical simulation broke down, usually in a region of high curvature; it signals that part of the posterior is not explored reliably. Reparameterize, use more informative priors for scale parameters, or increase `target_accept`, then check that the divergences disappear.

5. Why does HMC usually mix much better than random-walk Metropolis in high dimensions? Answer It uses gradient information to make long, directed moves that follow the shape of the posterior, so distant proposals still have a high acceptance probability. Random-walk proposals become almost always rejected unless they are tiny.

6. In a hierarchical SDT model, which parameter answers "how much do people differ in sensitivity?", and which answers "what is the typical sensitivity?" Answer σ_d (the group-level SD) and μ_d (the group-level mean), respectively.

7. List the steps of the Bayesian workflow. Answer Specify the model and priors $\rightarrow$ prior predictive check $\rightarrow$ fit with several chains $\rightarrow$ diagnose ($\hat{R}$, ESS, divergences, traces) $\rightarrow$ posterior predictive check $\rightarrow$ compare models if needed $\rightarrow$ interpret and report.

1.1.10 Assignment

- A5 (MCMC from scratch) is due at the start of the Week 14 lecture.
- Project progress check this week (see above).
- No new graded assignment: the remaining weeks are reserved for the final project.

1.1.11 Further reading

- Lee & Wagenmakers (2014), chapter 11. Signal detection theory, including the hierarchical SDT model you implemented today. *(Main reading.)*
- Kruschke (2014), chapter 9. Hierarchical models: partial pooling and shrinkage explained step by step.
- Farrell & Lewandowsky (2018), chapter 9. Multilevel or hierarchical modeling of cognitive data.
- Martin, Kumar & Lao (2021), chapters 2 and 10. Diagnostics with ArviZ, and probabilistic programming languages (how PyMC works under the hood).

Preview for Week 14: with the full toolkit in hand, we tour models from different areas of cognitive science, starting with categorization (the exemplar model GCM) and the speed–accuracy machinery of the drift-diffusion model.